

第八讲 信息系统的可靠性

一、信息系统可靠性

（一）信息系统可靠性的组成

➤ 计算机系统由硬件和软件组成，它们对整个系统的可靠性呈现完全不同的特性

➤ 计算机系统的可靠性研究基于硬件和软件的可靠性研究

硬件可靠性：硬件在使用过程中有磨损、材料的老化、变质和使用环境等多种因素

软件可靠性：软件使用期间无磨损，也不存在物质老化和变质

（二）提高信息系统可靠性的途径

提高可靠性有两个方面：一是尽量使系统在规定时间内少发生故障和错误；二是发生了故障能迅速排除。

硬件主要考虑如何提高元器件和设备的可靠性；采用抗干扰措施，提高系统对环境的适应能力和冗余结构设计。

软件主要考虑测试技术、故障自诊断技术、自动检错、纠错技术、系统恢复技术方面的设计。

（三）硬件可靠性

1、提高硬件可靠性

设计：结构优化，功能合理

环境：符合相关的标准和要求

使用：符合操作规程，减少人为失误

2、提高硬件可靠性的设计因素

(1) 设计

①结构优化设计

首先要根据性能指标和功能要求决定系统的结构形式，确定软硬件的分工和电路的具体结构。

系统的可靠性是由系统中的各个元件的可靠性决定的。只要能满足系统的性能指标，就应尽可能地简化系统结构，减少元件的数量，简化方案。

避免片面追求高性能指标和过多的功能。

合理划分软硬件功能，在 CPU 时间资源允许的前提下能够方便地用软件完成的功能一定要用软件实现。

②热设计

许多元器件的失效与温度有密切的关系。

热设计的正确与否是影响系统工作稳定性及可靠性的主要因素之一。

③机械防震设计

振动会对系统的元器件及整机结构造成机械性损坏，因此必须有机械防震设计。

重点在于系统接插件、系统模块和质量较大器件的固定。

④工艺设计

元器件的引脚焊点、模块间的接插件、总线插件等的电器互连是电子电路中故障率较高的部分，需要高度重视。

⑤环境防护设计

根据系统可能工作的环境进行防护设计，通常需要考虑温度、湿度、气压、雨、雪、盐雾、腐蚀性气体、沙尘及辐射等。

⑥抗干扰设计

抗干扰历来是硬件设计中一个伤脑筋的问题。干扰源、传输途径及干扰对象是构成干扰的三个要素。

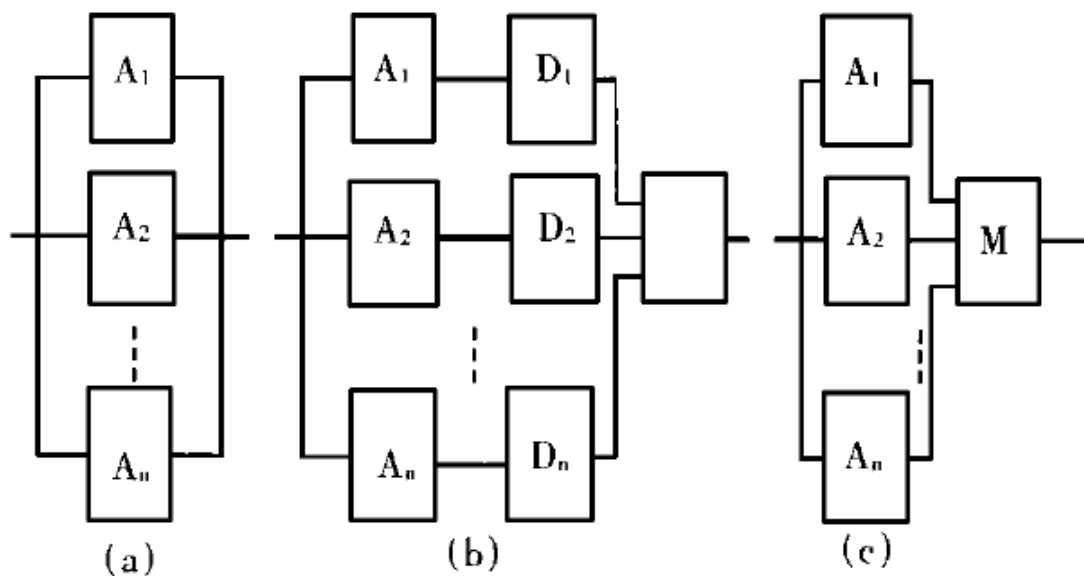
常用的抑制干扰的措施主要有：滤波、接地、屏蔽、隔离、设置干扰吸收网络及合理布线等。

采用合理的接地技术、屏蔽技术、隔离技术、以及长线传输、电源干扰的抑制等手段提高系统的抗干扰能力。

⑦冗余设计

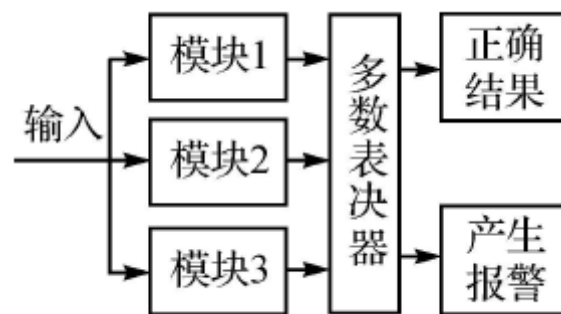
故障容错是利用冗余的元部件来屏蔽已发生的故障对系统的影响。

常用的冗余系统，按其结构可分为并联系统、备用系统和表决系统三种。



三模冗余 (TMR, Triple Modular Redundancy) 的基本原理是：系统输入通过 3 个功能相同的模块，产生的 3 个结果送到多数表决器进行表决，即三中取二的原则，如果模块中有一个出错，而另外两个模块正常，则表决器的输出正确，从而可以屏蔽一个故障。

TMR 的缺点是：如果 3 个模块的输出各不相同，则无法进行多数表决；若有两个模块出现一致的故障，则表决的结果会出现错误。



由于三模冗余技术简单性以及高可靠性，它是一个被广泛使用的针对于 FPGA 上的单粒子翻转 (Single-Event Upset, SEU) 的容错技术。

(2) 环境

① 温度变化对电容的影响

温度变化将引起电容的到介质损耗变化，从而影响其使用寿命。

- 温度每升高 10°C 时，电容器的寿命就降低 50%，同时还引起阻容时间常数变化，甚至发生因介质损耗过大而热击穿的情况；

- 温度升高也将使电感线圈、变压器、扼流圈等的绝缘性能下降。

②温度变化对半导体器件的影响

$$I_{CQ} = I_{CQR}[e^{0.069(T-T_R)}]$$

I_{CQ} : 温度 $T^{\circ}\text{C}$ 时的反向漏电流;

I_{CQR} : 温度 $T_R^{\circ}\text{C}$ 时的反向漏电流;

$T - T_R$: 温度变化的绝对值。

温度每升高 10°C , I_{CQ} 将增加一倍。这将造成晶体管放大器的工作点发生漂移、晶体管电流放大系数发生变化、特性曲线发生变化, 动态范围变小。

③温度与允许功耗的关系

$$P_{CM} = \frac{T_{JM} - T}{R_T}$$

P_{CM} : 最大允许功耗;

T_{JM} : 最高允许结温;

T : 使用环境温度;

R_T : 热阻。

温度的升高将使晶体管的最大允许功耗下降。

④温度变化对电阻的影响

温度变化对电阻的影响主要是温度升高时, 电阻的热噪声增加, 阻值偏离标称值, 允许耗散功率下降等。

比如, RXT 系列的碳膜电阻在温度升高到 100°C 时, 允许的耗散功率仅为标称值的 20%。但我们也可以利用电阻的这一特性, 比如经过特殊设计的一类电阻: PTC (正温度系数热敏电阻) 和 NTC (负温度系数热敏电阻), 它们的阻值受温度的影响很大。对于

PTC，当其温度升高到某一阈值时，其电阻值会急剧增大。利用这一特性，可将其用在电路板的过流保护电路中，当由于某种故障造成通过它的电流增加到其阈值电流后，PTC 的温度急剧升高，同时其电阻值变大，限制通过它的电流，达到对电路的保护。而故障排除后，通过它的电流减小，PTC 的温度恢复正常，同时其电阻值也恢复到其正常值。对于 NTC，它的特点是其电阻值随温度的升高而减小。

⑤湿度影响

湿度过高，当含有酸碱性的灰尘落到电路板上时，将腐蚀元器件的焊点与接线处，造成焊点脱落，接头断裂。湿度过高也是引起漏电耦合的主要原因。

湿度过低又容易产生静电，所以环境的湿度应控制在合理的水平。

⑥电压影响

- 施加在元器件上的电压稳定性是保证元器件正常工作的重要条件。过高的电压会增加元器件的热损耗，甚至造成电击穿。
- 对于电容器而言，其失效率正比于电容电压的 5 次幂。
- 对于集成电路而言，超过其最大允许电压值的电压将造成器件的直接损坏。

⑦振动、冲击影响

- 机械振动与冲击会使一些内部有缺陷的元件加速失效，造成灾难性故障；

- 机械振动还会使焊点、压线点发生松动，导致接触不良；
- 若振动导致导线不应有的碰连，会产生一些意想不到的后

果。

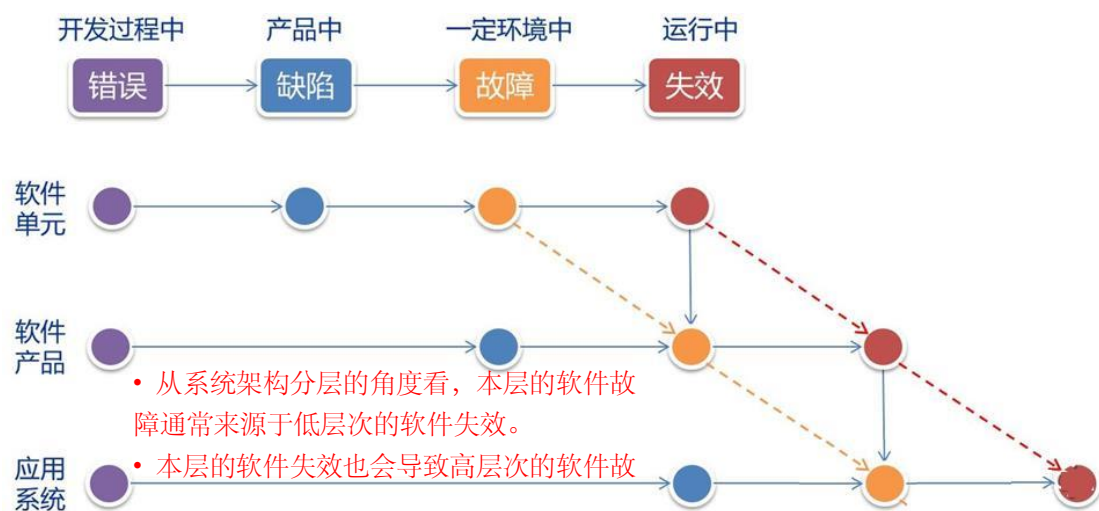
(3) 使用

Swain 定义人为失误为任何超过系统正常工作所能接受的标准与范围的人的行为与动作。

(四) 软件可靠性

1、软件失效机理

在软件开发过程中的疏忽、失误或者错误，导致软件中存在着缺陷；在软件运行过程中，当缺陷被激活（软件运行于某一特定条件，俗称“踩雷”），使软件产生错误状态即出现了故障；而若无适当的处理措施（容错）对故障加以处理，最终使软件无法向用户提供其所期望的服务，即产生了失效。



2、软件可靠性的定义

(1) 《GB/T 11457-2006 信息技术 软件工程术语》的定义

在规定条件下、在规定时间内，软件不引起系统失效的概率。
该概率是系统输入和系统使用的函数，也是软件中存在的缺陷的函数。系统输入将确定是否会遇到已存在的缺陷（如果有缺陷存在的话）；

在规定的周期内所述条件下程序执行所要求的功能的能力。

(2) 《GB/T 16260.1-2006 软件工程 产品质量 第1部分：质量模型》的定义

作为软件六大质量特性之一，软件可靠性的定义是：在指定条件下使用时，软件产品维持规定的性能级别的能力。

3、提高软件可靠性

(1) 采用系统信息管理软件

- 当发生故障时进行现场保护，迅速用备用系统代替故障系统；
- 在过负荷时采取应急措施；
- 在故障排除后使系统迅速恢复正常运行。

(2) 编制诊断程序



(3) 程序运行监视系统

程序运行监视系统 WTD（“看门狗”），是一种软硬件结合的抗程序“跑飞”的措施。其硬件是一个用于产生定时 T 的计数器或单

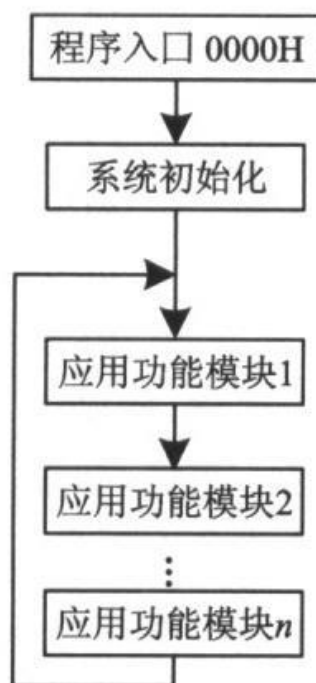
稳触发器。其定时输出端接至 CPU 的复位线，而其定时清零则由 CPU 控制。

在正常情况下，程序启动 WTD 后，即以小于 T 的间隔 t 将其清零一次，这样 WTD 的定时溢出就不会发生。在受到干扰的异常情况下，程序的正常执行顺序被破坏，不可能周期性地将 WTD 清零，导致 WTD 定时溢出，使系统复位，CPU 摆脱因程序“跑飞”造成的瘫痪状态。

WTD 是一种被动的抗干扰措施，它只能在一定程度上减少干扰造成的损失。其困难之处在于时间 T 长短的选择较困难。

看门狗的例子： AlwaysUp

4、软件陷阱



软件陷阱是指令冗余的一种应用形式，用于捕捉“跑飞”的指令指针。

➤ 对于因受干扰而混乱的程序，多字节指令是最危险的，因为错误的 PC 指针很有可能落在多字节指令的中间，造成指令错误执行；而单字节指令则可使混乱的 PC 指针重新理顺，使混乱现象得以控制。

➤ 软件陷阱就是根据这个道理构成的一个程序段。

以下是与 MCS-51 单片机相对应的一个软件陷阱：

```
NOP 00
```

```
NOP 00
```

```
LJMP ERROR 02 XXXX
```

程序中 LJMP ERROR 将“跑飞”的程序指针转移到出错处处理程序。其 NOP 指令加得越多，捕捉能力越强。

软件陷阱通常安排在下列位置：

- ．系统中空的 ROM 区。
- ．数据表格的头尾处。
- ．程序中未用的中断向量处。
- ．程序内如跳转指令等语句之后。

软件陷阱对陷入死循环的“跑飞”程序无能为力，WTD 这方面则更可靠，可以将两者结合使用。

5、指令复执技术

即在程序执行过程中，一旦发现错误就重新执行被错误干扰的现行指令。

指令复执既可用软件实现，也可以硬件实现。

但须遵循以下原则：

- a、发现错误时应能准确保留现行指令地址，以便重新执行。
- b、应能保留现行指令所用数据，以便重新执行使用。

二、软件可靠性模型

1、失效时间间隔模型

这类模型最常用的方法是假定第 i 个失效到第 $i+1$ 个失效间隔时间服从某一分布，而分布的参数依赖于各间隔时间内程序中的残留错误数。通过测试所得到的失效间隔时间数据来估计模型的参数，由获得的模型可以估算软件的可靠度以及各失效间的平均工作时间等；

另一种方法是把失效间隔时间看作随机过程，然后通过适当的时间序列模型来分析和描述软件的失效过程。这类模型有 Jelinski-Moranda 的非增长模型、Schick-Wolverton 模型、Littlewood-Verrall 的贝叶斯模型和 Goel-Okumoto 的不完善模型等。

2、缺陷计数模型

这类模型关心的是在**特定的时间间隔内软件的错误数或失效数**，并假定故障累计数服从某个已知的随机过程，过程强度是时间的离散或连续函数，根据在给定的测试时间间隔发现的错误数或失效数来估计故障强度、均值等参数。随着错误的不断排除，在单位时间内发现的失效数将不断减少。当时间间隔事先确定时，在每个间隔时间内的失效次数就是随机变量。这类模型有 Shooman 模型、Musa 的执行时间模型、Goel-Okumoto 的 NHPP 模型、Goel 的 HPP 模型和 Musa-Okumoto 的对数泊松执行时间模型等等。

3、错误植入模型

这类模型的基本思路是通过将一组已知的错误人为地植入到一个固有错误总数尚不清楚的程序中，然后在程序的测试中观察并统计发现的植入错误数和程序总的错误数，通过计数的比值估计程序

的固有错误总数，从而得到软件可靠度及其有关指标。这类模型的代表是 Mills 的超几何分布模型。

4、基于输入域的模型

这类模型的基本研究方法是根据程序的使用情况，找出程序可能输入的概率分布，根据这种分布产生一个测试用例的集合。由于得到输入的分布难度较大，一般将输入域划分成等价类，每个等价类与程序的一条执行路径相联。在输入域上随机抽取测试用例，执行相应的程序测试，观测故障，从而推断出各项指标。这类模型以 Nelson 模型为代表，还有 Brown-Lipow 的基于输入域模型和 Ramamoorthy-Bastani 的基于输入域模型。